МИНИСТЕРСТВО ЦИФРОВЫХ ТЕХНОЛОГИЙ РЕСПУБЛИКИ УЗБЕКИСТАН
ТАШКЕНТСКИЙ УНИВЕРСИТЕТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ
имени Мухаммада ал-Хоразмий

Дисциплина: «Введение в программную инженерию»

ПРАКТИЧЕСКАЯ РАБОТА № 6

Модель проектирования системы: архитектура, компоненты и UML-диаграммы

Вариант 19: разработка АРМ администратора салона красоты

Выполнил: **Тожикулов Асилбек**
Группа: **M320-24**
Вариант: **19**
Принял: _____

Ташкент, 2026

1. Тема практической работы

Модель проектирования системы: архитектура, компоненты и UML-диаграммы.

Цель работы: закрепление знаний по теоретическим сведениям. Научиться строить модель проектирования программной системы: декомпозировать систему на подсистемы, выбирать и обосновывать архитектурный стиль, строить UML-диаграммы компонентов, проектных классов и развёртывания, оценивать связность и сцепление модулей.

Вариант 19: разработка АРМ администратора салона красоты (BeautyDesk). Исходные данные – первоначальная архитектура из ПР 3 и варианты использования UC-01 – UC-13, требования FR/NFR и диаграмма деятельности из ПР 5.

2. Описание системы и перечень подсистем

BeautyDesk – веб-система для администраторов сети салонов красоты: база клиентов, графики мастеров и справочник услуг, журнал записи без накладок, касса с Payme и Click, напоминания клиентам, учёт материалов, отчёты и расчёт зарплаты. Пользователи работают через браузер, сервер и база данных размещены в дата-центре в Ташкенте. По вариантам использования из ПР 5 система разделена на 6 прикладных подсистем и 4 инфраструктурных модуля.

Таблица 1 – Прикладные подсистемы

Подсистема	Назначение	Варианты использования	Предоставляемый интерфейс
Клиенты	база клиентов, поиск по телефону, согласие на обработку данных, история визитов	UC-02, UC-07	IClientLookup: findByPhone(), getCard(), register()
Расписание	справочник услуг, графики смен и перерывы мастеров	UC-10, UC-11, UC-13	IScheduleProvider: shiftOf(), breaksOf(), durationOf()
Журнал записи	создание, перенос и отмена записей, проверка пересечений, календарь дня	UC-01, UC-03, UC-06	IBookingService: book(), reschedule(), cancel(), calendar()
Касса	оплата визита, фискальный чек, смена кассы, списание материалов	UC-04, UC-05, UC-08	IPaymentService: pay(), openShift(), closeShift()
Уведомления	сообщения и напоминания клиентам в Telegram и SMS	UC-09	INotifier: notify(), scheduleReminder()
Отчёты и зарплата	отчёты по выручке и загрузке, расчёт зарплаты мастеров	UC-12	IReportService: revenue(), workload(), payroll()

Таблица 2 – Инфраструктурные модули

Инфраструктурный модуль	Назначение	Интерфейс
API и аутентификация	REST-точка входа, JWT-сессии, проверка ролей, журнал действий	HTTP REST /api/v1
Доступ к данным	репозитории над PostgreSQL, миграции	IRepository и его специализации
Блокировка слотов	распределённая блокировка времени мастера при записи	ISlotLock: acquire(), release()
Адаптеры интеграций и планировщик	Payme, Click, онлайн-касса, SMS, Telegram; фоновые задачи Celery	IPaymentGateway, ISmsGateway, IJobScheduler

3. Архитектурный стиль и его обоснование

Выбран **многоуровневый (слоистый) модульный монолит**: уровни представления (SPA в браузере), приложения (подсистемы на сервере) и данных; внутри уровня приложения код разделён на модули по предметным областям, которые общаются только через интерфейсы из таблицы 1. Такой стиль был намечен в ПР 3, здесь он уточнён до уровня подсистем и интерфейсов. Для сравнения рассмотрены **микросервисная архитектура** и **настольное клиент-серверное приложение с базой в каждом салоне**.

Таблица 3 – Обоснование архитектурного стиля по FURPS

Атрибут FURPS / требование	Ключевое требование	Модульный монолит	Микросервисы	Настольное + локальная БД
F – Functionality	FR-05: запись без накладок, в т. ч. одновременная	5 – одна транзакционная БД + блокировка	3 – нужны распределённые транзакции	2 – нет общей картины между салонами
U – Usability	NFR-05: работа из любого салона без установки, RU/UZ	5 – браузер	5 – браузер	2 – установка и обновление на каждом ПК
R – Reliability	NFR-02: 99 % доступности, резервные копии	4 – один сервер + ежедневный бэкап	4 – выше отказоустойчивость, но больше точек отказа	3 – копии в каждом салоне

Атрибут FURPS / требование	Ключевое требование	Модульный монолит	Микросервисы	Настольное + локальная БД
P – Performance	NFR-01: календарь ≤ 2 с, 30 пользователей	5 – нагрузка мала, вызовы внутри процесса	4 – сетевые вызовы между сервисами	5 – локально быстро
S – Supportability	три инкремента, команда из 3 разработчиков, NFR-06	5 – один репозиторий и развёртывание, модули расширяются	2 – CI/CD и оркестрация для каждого сервиса	2 – обновление на всех ПК
+ ограничения	бюджет ≈ 135 млн сум, данные в Узбекистане (NFR-04)	5	2	3
Итого (из 30)		29	20	17

Модульный монолит лучше всего отвечает ключевому требованию FR-05: проверка пересечения и сохранение записи выполняются в одной транзакции одной базы данных. Микросервисы дали бы независимое масштабирование, но для трёх салонов и 30 пользователей оно не нужно, а цена – распределённые транзакции и инфраструктура, которой нет у небольшой команды. Настольное приложение не обеспечивает общей базы клиентов и отчётов по сети. Чёткие границы модулей оставляют возможность при росте вынести, например, «Уведомления» в отдельный сервис без переписывания остальных.

4. Диаграмма компонентов и диаграмма проектных классов

4.1. Диаграмма компонентов

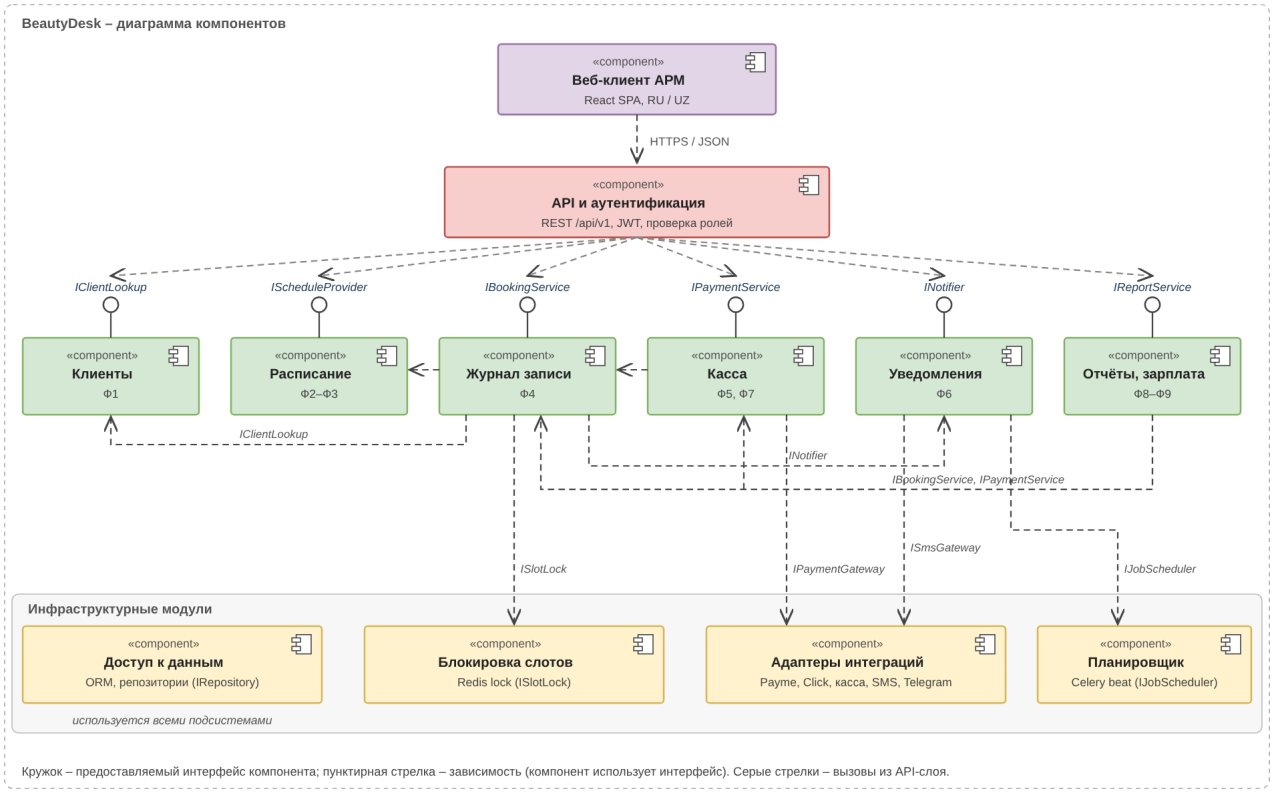


Рисунок 1 – Диаграмма компонентов системы BeautyDesk

Веб-клиент обращается только к компоненту «API и аутентификация» по HTTPS; API проверяет роль пользователя и вызывает подсистемы через их интерфейсы (кружки на диаграмме). Между подсистемами зависимости направлены так, чтобы не было циклов: «Журнал записи» использует «Расписание», «Клиентов» и «Уведомления»; «Касса» использует «Журнал записи»; «Отчёты» только читают данные «Журнала» и «Кассы». Подсистемы обращаются к внешним сервисам не напрямую, а через адаптеры интеграций, поэтому смена платёжного провайдера затронет только один модуль.

4.2. Диаграмма проектных классов подсистемы «Журнал записи»

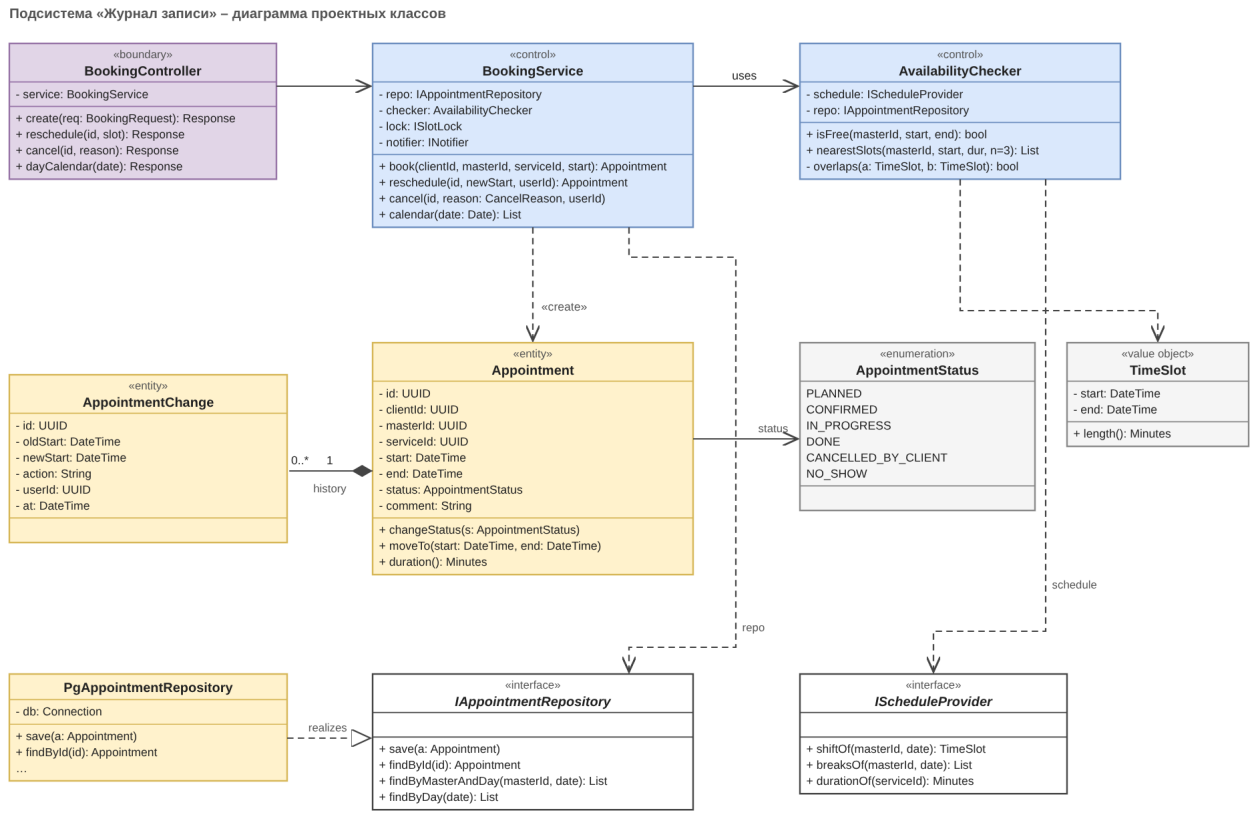


Рисунок 2 – Диаграмма проектных классов подсистемы «Журнал записи»

Таблица 4 – Проектные классы подсистемы «Журнал записи»

Класс	Тип	Пояснение
BookingController	boundary	принимает HTTP-запросы от API, проверяет входные данные, вызывает сервис и возвращает ответ
BookingService	control	координирует сценарии UC-01 и UC-03: блокирует слот (ISlotLock), проверяет доступность, создаёт или переносит запись, сохраняет историю и отправляет уведомление (INotifier)
AvailabilityChecker	control	проверяет смену и перерывы мастера через IScheduleProvider, ищет пересечения интервалов и 3 ближайших свободных окна (FR-04, FR-05)
Appointment	entity	запись клиента: мастер, услуга, время, статус; изменяет статус и время только через свои операции
AppointmentChange	entity	история изменений записи (FR-06); связана с Appointment композицией 1 – 0..*
AppointmentStatus	enumeration	6 статусов из FR-07, включая CANCELLED_BY_CLIENT и NO_SHOW
TimeSlot	value object	неизменяемый интервал времени, используемый в проверке пересечений

Класс	Тип	Пояснение
IAppointmentRepository, PgAppointmentRepository	interface / класс	доступ к данным записей; реализация на PostgreSQL. Сервис зависит от интерфейса, поэтому в тестах репозиторий заменяется заглушкой
ISchedule Provider	interface	требуемый интерфейс подсистемы «Расписание»; «Журнал» не знает, как хранятся смены

5. Оценка связности и сцепления модулей

Связность (cohesion) оценивалась по шкале от случайной до функциональной: насколько элементы модуля служат одной задаче. **Сцепление** (coupling) – по типу и числу зависимостей модуля от других: от сцепления по данным (лучшее) до сцепления по содержимому (худшее). Число зависимостей подсчитано по диаграмме компонентов (Ca – входящие, Ce – исходящие).

Таблица 5 – Оценка связности и сцепления модулей

Модуль	Связность	Ca	Ce	Тип сцепления	Проблема / рефакторинг
Клиенты	функциональная – высокая	2	1	по данным (через IClientLookup)	нет
Расписание	коммуникационная – средняя: услуги и смены работают с разными данными	2	1	по данным	разделить на «Каталог услуг» и «Смены мастеров», когда появятся цены по салонам
Журнал записи	функциональная – высокая	3	5	по данным; Ce самое высокое среди подсистем	проблемное место: зависит от 5 модулей. Отправку уведомлений перевести на события «запись создана / изменена», чтобы Журнал не вызывал INotifier напрямую (Ce: 5 → 4)
Касса	последовательная – средняя: оплата, чек и списание материалов	2	3	по данным; сцепление по общей области – прямое чтение таблицы записей	проблемное место: читать визиты только через IBookingService; списание материалов вынести в отдельный модуль «Склад» (Ф7)
Уведомления	функциональная – высокая	2	3	по данным	нет

Модуль	Связность	Ca	Ce	Тип сцепления	Проблема / рефакторинг
Отчёты и зарплата	логическая – низкая: разные отчёты и расчёт зарплаты объединены по признаку «вывод данных»	1	3	по данным, только чтение	выделить «Расчёт зарплаты» в отдельный модуль; отчёты строить по реплике/представлениям БД
API и аутентификация	функциональная	1	6	по управлению (маршрутизация)	допустимо – это точка входа
Адаптеры интеграций	логическая – низкая: разные внешние системы в одном модуле	2	0	по данным	разбить на «Платежи», «Касса ОФД» и «Сообщения» с общим базовым интерфейсом

Итог оценки. Большинство модулей имеют функциональную связность и сцепление только по данным через интерфейсы – это хороший уровень для модульного монолита. Выявлены три проблемных места: (1) высокое исходящее сцепление «Журнала записи», (2) сцепление «Кассы» с «Журналом» через общую таблицу, (3) логическая связность модулей «Отчёты» и «Адаптеры». Предложенный рефакторинг – события вместо прямых вызовов уведомлений, доступ к данным визитов только через интерфейс и разделение модулей с низкой связностью – уменьшает число межмодульных зависимостей и упрощает последующий вынос модулей в отдельные сервисы. Рефакторинг планируется на инкремент 2, когда реализуется «Касса».

6. Диаграмма развёртывания

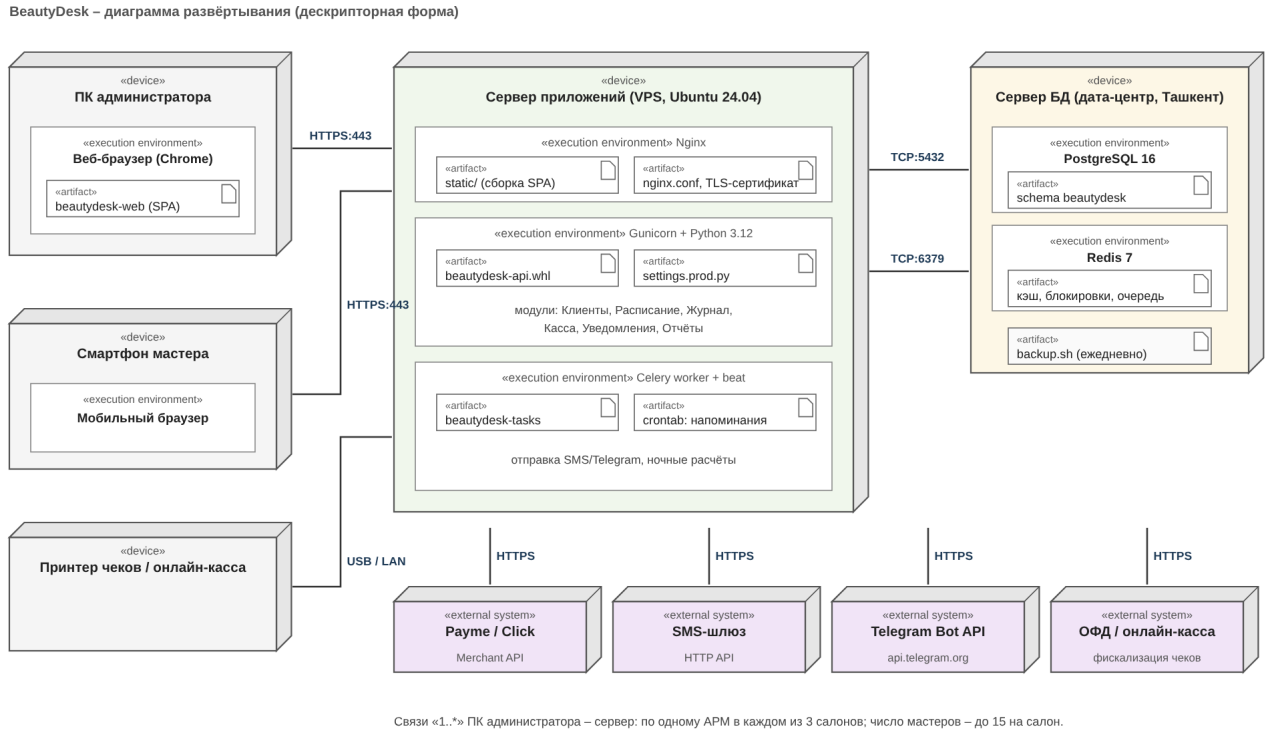


Рисунок 3 – Диаграмма развёртывания системы BeautyDesk (дескрипторная форма)

Диаграмма построена в **дескрипторной форме**: показаны типы узлов, а не конкретные экземпляры (на практике АРМ установлены в каждом из трёх салонов). Узлы и размещённые артефакты:

- **ПК администратора** и **смартфон мастера** – браузер, в котором выполняется артефакт beautydesk-web (SPA); соединение с сервером – HTTPS, порт 443;
- **Сервер приложений** (VPS, Ubuntu 24.04) – три среды выполнения: Nginx (статика SPA, TLS), Gunicorn + Python 3.12 (артефакт beautydesk-api.whl со всеми подсистемами) и Celery (фоновые задачи: напоминания, ночные расчёты);
- **Сервер БД** в дата-центре в Ташкенте – PostgreSQL 16 (схема beautydesk, TCP 5432) и Redis 7 (кэш, блокировки слотов, очередь задач, TCP 6379), а также скрипт ежедневного резервного копирования; размещение в Узбекистане выполняет требование NFR-04;
- **Внешние системы** – Payme/Click, SMS-шлюз, Telegram Bot API, оператор фискальных данных; все соединения по HTTPS;
- **Принтер чеков / онлайн-касса** подключается к ПК администратора по USB или LAN.

Порты БД открыты только для сервера приложений, наружу открыт лишь порт 443 Nginx, что соответствует требованию безопасности NFR-03.

7. Вывод о проделанной работе

В работе построена модель проектирования системы «АРМ администратора салона красоты». На основе вариантов использования и аналитической модели из ПР 5 система декомпозирована на 6 прикладных подсистем (Клиенты, Расписание, Журнал записи, Касса, Уведомления, Отчёты и зарплата) и инфраструктурные модули; для каждой подсистемы указаны назначение, поддерживаемые варианты использования и предоставляемый интерфейс.

Выбран многоуровневый модульный монолит; по атрибутам качества FURPS и ограничениям проекта он набрал 29 баллов из 30 против 20 у микросервисов и 17 у настольного приложения, главным образом за счёт гарантии записи без накладок в одной транзакционной базе и простоты сопровождения для небольшой команды. Построены диаграмма компонентов с интерфейсами и зависимостями без циклов и диаграмма проектных классов подсистемы «Журнал записи» с типами классов boundary, control, entity, interface и value object.

Оценка связности и сцепления показала, что большинство модулей функционально связны и сцеплены только по данным; выявлены три проблемных места и предложен рефакторинг (события вместо прямых вызовов уведомлений, доступ к визитам только через интерфейс, разделение модулей с логической связностью). Построена диаграмма развёртывания в дескрипторной форме с узлами, артефактами и протоколами соединений.